Chapter 11

Caching

Why this chapter matters: Chapter 2 showed that memory is ~1,000× faster than SSD and a data-centre round trip costs 0.5 ms; Chapters 9–10 showed that databases are the hardest tier to scale. **Caching** is the cheapest way to exploit the gap: keep the answers to frequent questions in fast memory close to the caller. It improves latency, cuts load on servers and databases, and absorbs traffic spikes — but it adds the notoriously hard problem of **keeping copies correct**. This chapter covers where to cache, what to cache, how to update and evict, and how caches fail.

> ⚡ **QUICK RECALL — Chapter map**
> - 11.1 Why caching works → 11.2 Cache layers & what to cache → 11.3 Update strategies (cache-aside, read/write-through, write-behind, refresh-ahead)
> - 11.4 Eviction & expiry → 11.5 Invalidation & consistency → 11.6 Failure modes (stampede, penetration, avalanche, hot keys)
> - 11.7 Distributed caching: Redis vs Memcached, sharding, HA → 11.8 Common cache patterns → 11.9 Disadvantages & when not to cache
> - 11.10 Worked examples → 11.11 Practice, Q&A, cheat sheet

11.1 Why caching works

Caching improves page load times and can reduce the load on your servers and databases. In the dispatcher model, the dispatcher first looks up whether the request has been made before and tries to find the previous result to return, in order to save the actual execution:

```
Client --> [ Dispatcher ] --1) lookup--> [ Cache ]
                ^   |                        |
                |   |<---- 2) found: return ---+      (a HIT: fast, no backend work)
                |   |
                |   +--3) not found: forward to a worker, then store the result in the cache
                |          [ Worker pool / DB ]
                +----------- response back to the client
```

- **Locality** (Chapter 2): a small fraction of data gets most of the traffic (≈ 80/20), so a small cache can absorb most reads.
- **Skew absorption**: databases often benefit from a uniform distribution of reads and writes across their partitions, but **popular items can skew the distribution**, causing bottlenecks. Putting a cache in front of a database helps **absorb uneven loads and spikes in traffic**.
- **Cost**: results are recomputed far less often — saving CPU, DB I/O and money.

Hit ratio is what matters

Average access time = hit% × cache time + miss% × (cache time + backend time). With cache 1 ms and database 20 ms (a miss pays both):

Hit ratio	Average latency	Backend load at 20,000 req/s
0% (no cache)	20 ms	20,000 req/s
80%	0.8×1 + 0.2×21 = **5.0 ms**	4,000 req/s
90%	0.9×1 + 0.1×21 = **3.0 ms**	2,000 req/s
99%	0.99×1 + 0.01×21 = **1.2 ms**	**200 req/s**
99.9%	≈ 1.02 ms	20 req/s

★ **MUST REMEMBER**

- Think in **miss rate**: going from 90% to 99% hits makes the backend see **10× less traffic**. A cache only helps if the hit ratio is high — which needs **repeated access**, **enough memory** for the hot set, and **sensible TTLs**.
- **A cache is an optimisation, never the source of truth.** The system must still be correct (if slower) when the cache is empty, stale or down.

11.2 Cache layers & what to cache

Caches can be located on the client side (OS or browser), server side, or in a distinct cache layer. A single request may pass through many caches:

```
Browser/OS cache -> CDN edge -> Reverse proxy / web-server cache -> App in-process cache
   -> Distributed cache (Redis / Memcached) -> DB buffer pool / DB caches -> disk
(each layer: faster, closer to the user, smaller hit-set; every layer can serve a stale copy)
```

Layer	What / how	Examples & notes
Client caching	Browser cache, OS DNS cache, mobile app cache; controlled by HTTP headers (Cache-Control, ETag — Chapter 4)	Zero network cost; you cannot purge it → use versioned URLs
CDN caching	CDNs are considered a type of cache (Chapter 6)	Static assets, media, cacheable API responses
Web-server caching	**Reverse proxies and caches such as Varnish can serve static and dynamic content directly.** Web servers can also cache requests, returning responses without having to contact application servers	Nginx `proxy_cache`, Varnish; micro-caching 1–5 s absorbs spikes
Database caching	Your database usually includes some level of caching in a default configuration, **optimised for a generic use case**. Tweaking these settings for specific usage patterns can further boost performance	Buffer pool / shared_buffers sizing, OS page cache (Chapter 2). MySQL's query cache was removed in 8.0 (Chapter 9)
Application caching	**In-memory caches such as Memcached and Redis are key-value stores between your application and your data storage.** Since data is held in RAM it is much faster than typical databases where data is stored on disk	The workhorse of system design; the focus of this chapter

In-process (local) vs distributed cache

	In-process (L1)	Distributed (L2)
Where	Inside each app instance's memory (Caffeine, Guava, `lru-cache`, Map)	Separate cluster (Redis, Memcached) shared by all instances
Latency	**Nanoseconds–µs** (no network)	~0.2–1 ms network round trip (Chapter 2)
Consistency	Each instance holds its own copy → **copies diverge**; invalidation must be broadcast or TTL very short	One shared copy → consistent across instances
Capacity	Limited by instance RAM; adds GC pressure (Node/JVM)	Scales out by sharding
Typical use	Tiny hot sets, config, reference data, per-request memoisation	Sessions, profiles, query results, rendered fragments

◆ **SENIOR INSIGHT — Two-level caching (L1 + L2)**

- Combine a tiny, short-TTL in-process cache (seconds) in front of the distributed cache: it removes the network hop for the hottest keys and protects Redis from hot-key overload (11.6). Accept slightly staler data on L1 and invalidate via pub/sub if needed.

What granularity to cache

Level	Key → value	Pros / cons
Row level	`user:42` → one row	Simple, easy to invalidate; many small lookups
Query level	hash(SQL text) → result set	Easy to add; **expiration issues** (below)
Fully-formed serialisable objects	`profile:42` → assembled object (user + settings + counts)	**Best balance**: saves joins and assembly; one key per business entity
Fully-rendered HTML / fragments	`page:/home:v3` → HTML	Largest saving; hardest to personalise/invalidate

✖ COMMON MISTAKE — **Avoid file-based caching**

- Generally you should try to **avoid file-based caching, as it makes cloning and auto-scaling more difficult**: each server gets its own private files that others can't invalidate or share (stateless servers, Chapter 5). Use a shared in-memory cache instead.

Caching at the database query level vs the object level

Query level. Whenever you query the database, **hash the query as a key and store the result** in the cache. This approach suffers from expiration issues: it is **hard to delete a cached result with complex queries**, and if **one piece of data changes (such as a table cell) you need to delete all cached queries that might include the changed cell**.

Object level. See your data as an **object**, similar to what you do with your application code. Have your application assemble the dataset from the database into a class instance or a data structure; then **remove the object from the cache if its underlying data has changed**. This also **allows for asynchronous processing: workers assemble objects by consuming the latest cached object**.

★ MUST REMEMBER — **Suggestions of what to cache**

- **User sessions** • **fully rendered web pages** (or fragments) • **activity streams** (timelines/feeds) • **user graph data** (followers/friends) • plus: product/catalogue entries, permissions, config/feature flags, expensive aggregates, results of third-party calls, rate-limit counters, and negative results ("not found").

11.3 When to update the cache: the strategies

Since you can only store a limited amount of data in the cache, you need to determine which cache-update strategy works best for your use case.

1) Cache-aside (lazy loading)

The application is responsible for reading and writing from storage. The cache does not interact with storage directly. The application: looks for the entry in the cache (a miss) → loads it from the database → adds it to the cache → returns it. Memcached is generally used in this manner.

```
App --1) GET key--> [ Cache ]      HIT  -> return immediately
 |                     | MISS
 +--2) SELECT ... ----> [ Database ]
 +--3) SET key,value,TTL -> [ Cache ]      4) return the value
```

```
// cache-aside, corrected from the pseudo-code in your notes
async function getUser(id) {
  const key = `user:${id}`;                          // ONE key, used for both get and set
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);             // HIT
  const user = await db.query('SELECT * FROM users WHERE id = $1', [id]);   // MISS -> database
  if (user) await redis.set(key, JSON.stringify(user), { EX: 3600 });       // populate, with a TTL
  return user;
}
```

> ✎ **CORRECTION TO YOUR ORIGINAL NOTE — Your Python cache-aside example**
>
> • The sketch builds the key inconsistently (`cache.get("user.{0}", user_id)` vs `key = ...format(user_id)`) and passes parameters in a way no client accepts. Always compute **one key** and reuse it for get and set, and use parameterised SQL. The pattern itself is correct.

- **Pros**: subsequent reads of data added to the cache are fast. Cache-aside is also referred to as **lazy loading**: only requested data is cached, which avoids filling the cache with data that isn't requested. Resilient: if the cache is down, the app still works from the DB.
- **Cons**: **each cache miss results in three trips** (cache read, DB read, cache write), which can cause a noticeable delay; data can become **stale** if it is updated in the database (mitigate with a **TTL**, invalidation, or write-through); **when a node fails, it is replaced by a new, empty node, increasing latency** (cold start).

2) Read-through

The application talks **only to the cache**; on a miss the **cache itself loads the data** from the database (a loader function is configured in the cache library or service, e.g., Caffeine *LoadingCache*, DynamoDB DAX). Same effect as cache-aside, but the loading logic lives in the cache layer — simpler call sites, less flexibility.

3) Write-through

The application uses the cache as the main data store, reading and writing data to it, while the cache is responsible for reading and writing to the database: the app adds/updates the entry in the cache → the cache **synchronously** writes the entry to the data store → return.

```
   App --1) write--> [ Cache ] --2) synchronous write--> [ Database ]
                                          3) return to the app after BOTH succeed

   // Application code:   setUser(12345, {"foo": "bar"})
   // Cache layer logic (the notes' snippet omitted SET in the UPDATE):
   //    def set_user(user_id, values):
   //        db.update("UPDATE users SET ... WHERE id = %s", ...)
   //        cache.set(user_id, user)
```

- Write-through is a **slow overall operation** due to the write, but **subsequent reads of just-written data are fast**. Users are generally more tolerant of latency when updating data than when reading it. **Data in the cache is not stale.**
- **Disadvantages**: when a **new node is created** due to failure or scaling, it will not cache entries until they are updated in the database — cache-aside in conjunction with write-through can mitigate this; **most data written might never be read**, which can be minimised with a TTL.

4) Write-behind (write-back)

The app adds/updates the entry in the cache and returns; the cache **asynchronously** writes the entry to the data store (via a queue and event processor), **improving write performance**. It can batch and coalesce many updates to the same key into one DB write.

```
App --1) write--> [ Cache ] --3) return to the user at once
                    |
                    +--2) add event--> [ Queue ] --> [ Event processor ]
                                                       |  4) asynchronously
                                                       v
                                                  [ Database ]
```

- **Disadvantages**: **there could be data loss if the cache goes down prior to its contents hitting the data store**; it is **more complex to implement** than cache-aside or write-through (ordering, retries, failure handling).
- Use for high-volume, loss-tolerant or reconstructable writes: counters, likes, view counts, metrics, last-seen timestamps.

5) Refresh-ahead

You can configure the cache to **automatically refresh any recently accessed cache entry prior to its expiration**, so hot keys never expire under load. Refresh-ahead can result in **reduced latency vs read-through** if the cache can **accurately predict which items are likely to be needed in the future**. **Disadvantage**: not accurately predicting can result in **reduced performance than without refresh-ahead** (wasted loads). Also known as 'background refresh' / 'stale-while-revalidate'.

6) Write-around

Writes go **straight to the database**, bypassing the cache; the cache is filled only by reads. Good when written data is rarely read soon (avoids polluting the cache) — but the first read after a write is a miss.

Strategy	DB access by	Write latency	Staleness	Data loss?	Effort	Best for
Cache-aside	Application	Normal (DB)	Until TTL / invalidation	None	Low	General default; read-heavy
Read-through	Cache	Normal	Until TTL	None	Low–medium	Same, with a caching library/service
Write-through	Cache (sync)	Higher (cache + DB)	None	None	Medium	Read-after-write correctness, reads of fresh data
Write-behind	Cache (async)	**Lowest**	DB lags the cache	**Yes** (cache crash)	High	Write-heavy, loss-tolerant, batching
Refresh-ahead	Cache	—	Minimal for hot keys	None	Medium	Predictable hot keys, strict read latency
Write-around	Application	Normal	Until TTL (first read refills)	None	Low	Write-once-read-rarely data

◆ **SENIOR INSIGHT** — **In practice: cache-aside + delete-on-write**

- Most production systems use **cache-aside for reads** and, on writes, **update the database first, then *delete* (not update) the cached key**; the next read repopulates it. A short TTL acts as a safety net.
- Why delete instead of set? Two concurrent writers can update the DB in order A, B but the cache in order B, A — leaving a **stale value in the cache until the TTL expires** (race below). Deleting is idempotent and removes this race.

```
Race 1 - 'update the cache on write':
  Writer A: UPDATE db = 1        Writer B: UPDATE db = 2        Writer B: cache.set(2)
  Writer A: cache.set(1)   ->  cache = 1 but db = 2   (stale until the TTL expires)

Race 2 - cache-aside + delete (rare, but real):
  Reader: cache MISS, reads db = v1 ...        Writer: UPDATE db = v2, DELETE cache key
  Reader: ... then SET cache = v1   ->  stale v1 cached.
  Mitigate: TTL, 'delayed double delete' (delete again after ~1 s),
            leases / versioned writes, or CDC-driven invalidation
```

11.4 Eviction & expiry

RAM is more limited than disk, so **cache invalidation algorithms such as least recently used (LRU) can help invalidate 'cold' entries and keep 'hot' data in RAM**. Two separate mechanisms remove data: **expiry** (TTL says it is too old) and **eviction** (memory is full, something must go).

Policy	Evicts	Good for	Notes
LRU (least recently used)	The entry untouched for the longest time	General workloads with temporal locality	O(1) with hash map + doubly linked list; a one-off scan can flush the hot set
LFU (least frequently used)	The entry with the fewest accesses	Stable popularity (top products)	Resists scan pollution; needs ageing so old favourites fade
TinyLFU / W-TinyLFU	Admission filter + LRU window	Best hit ratio in practice	Used by Caffeine
FIFO	Oldest inserted	Simple, predictable	Ignores usage
Random	A random entry	Very cheap	Surprisingly OK for large caches
TTL-based	Entries whose time-to-live elapsed	Freshness bound	Complements any eviction policy

```
class LRU {                                          // O(1) get/set using Map's insertion order
  constructor(capacity) { this.cap = capacity; this.map = new Map(); }
  get(k) {
    if (!this.map.has(k)) return undefined;
    const v = this.map.get(k);
    this.map.delete(k); this.map.set(k, v);          // move to the 'most recent' end
    return v;
  }
  set(k, v) {
    this.map.delete(k); this.map.set(k, v);
    if (this.map.size > this.cap) this.map.delete(this.map.keys().next().value);  // evict oldest
  }
}
```

- **Redis** `maxmemory-policy`: `noeviction` (writes fail when full — right for a primary store), `allkeys-lru` / `allkeys-lfu` (pure cache), `volatile-lru` / `volatile-ttl` (only keys with a TTL). Redis uses *approximate* LRU by sampling.
- **Always set a TTL** on cache entries: bounds staleness, reclaims memory and heals forgotten invalidations. Pick TTL per data type (seconds for prices, minutes for profiles, hours for catalogue).
- **Add jitter** to TTLs (e.g., 3600 ± 10%) so keys created together do not all expire together (avalanche, 11.6).

11.5 Invalidation & consistency — the hard part

"There are only two hard things in computer science: cache invalidation and naming things." You must **maintain consistency between caches and the source of truth** (the database) through cache invalidation, and there is additional complexity in **deciding when to update the cache**.

Technique	How	Trade-offs
TTL expiry	Entry disappears after N seconds	Simple; staleness bounded by N; wasteful reloads of unchanged data
Delete / update on write	Write path removes or rewrites the key after DB commit	Fresh quickly; every writer must know every key it affects
Event-driven (CDC / pub-sub)	DB changes stream (Debezium → Kafka) to an invalidator; or publish "user:42 changed" to all app instances	Decoupled writers; handles L1 caches; eventual and needs infrastructure
Versioned keys	Embed a version in the key: `user:42:v7`; bump version on change (stored in a small fast key)	No delete needed; old versions age out; extra lookup for the version
Tags / dependency sets	Group keys by tag (`tag:product:9` → set of keys) and delete the group	Handles query/page caches; bookkeeping overhead
Stale-while-revalidate	Serve stale, refresh in background	Great latency; deliberate staleness window

◆ **SENIOR INSIGHT** — **Design for a staleness budget, not perfection**

- State per data type how stale it may be: **strongly consistent** (account balance, permissions → don't cache, or read from primary), **seconds** (inventory display, prices), **minutes** (profiles, catalogue), **hours** (static reference data). The TTL *is* the contract.
- For read-your-writes (Chapter 5), after a user writes, **update or bypass** the cache for that user's next reads, or return the new object from the write response.
- Query-level caches are the hardest to invalidate (see 11.2) — prefer **entity/object caches** keyed by id, and compute lists/search results from ids + cached entities.
- **Security**: never cache authorisation decisions or personal data longer than allowed or under a shared key (include tenant/user in the key); cache poisoning and key-collision bugs leak data across users.

11.6 Failure modes of caching

Problem	What happens	Mitigations
Cache stampede (thundering herd, dog-pile)	A hot key expires; thousands of concurrent requests all miss and hit the database together	**Request coalescing / single-flight** (one loader, others wait); mutex/lease on the key; **stale-while-revalidate**; refresh-ahead; TTL jitter
Cache penetration	Requests for keys that **don't exist** (random ids, attacks) always miss and always reach the DB	**Cache null/negative results** briefly (30–60 s); validate ids; **Bloom filter** of existing keys; rate-limit
Cache avalanche	Many keys expire at once, or the whole cache fails → the database is overwhelmed and collapses	TTL jitter; multi-node HA cache; multi-level caches; **load-shed and circuit-break** DB access; gradual warm-up
Hot key	One key (a celebrity profile) gets more traffic than a single cache node can serve	Local L1 cache; **replicate the hot key** under N suffixed keys (`key#0..N`) and read a random copy; CDN for public data
Big key / slow command	Giant values or O(n) commands block a single-threaded node	Cap value sizes, split big structures, avoid KEYS/SMEMBERS on huge sets
Cold start	New/restarted node is empty → latency spike and DB load	Pre-warm hot keys; rolling restarts; persistence/replicas to refill; gradual traffic ramp
Inconsistent copies	Races in updates (11.3), failed invalidations	Delete-on-write, TTL safety net, CDC invalidation
Cache outage	Redis down → every request falls through to the DB	HA (replicas + failover); **degrade gracefully**: serve stale/limited features; protect the DB with limits

Cache-aside with single-flight, TTL jitter and negative caching (Node)

```
const inflight = new Map();                                  // key -> Promise (per process)

async function getUser(id) {
  const key = `user:${id}`;
  const hit = await redis.get(key);
  if (hit !== null) return JSON.parse(hit);                  // may be a cached 'null' (negative caching)

  if (inflight.has(key)) return inflight.get(key);           // coalesce concurrent misses: ONE DB call
  const p = (async () => {
    const user = await db.findUser(id);                      // null if the id does not exist
    const jitter = Math.floor(Math.random() * 360);          // +0..10% avoids synchronized expiry
    const ttl = user ? 3600 + jitter : 60;                   // short TTL for 'not found' (penetration)
    await redis.set(key, JSON.stringify(user), { EX: ttl });
    return user;
  })().finally(() => inflight.delete(key));
  inflight.set(key, p);
  return p;
}
```

11.7 Distributed caching: Redis, Memcached, sharding & HA

	Redis	Memcached
Data model	**Rich structures**: strings, hashes, lists, sets, **sorted sets**, streams, bitmaps, geo	Simple strings/blobs (key → value)
Persistence	**Optional** (RDB snapshots, AOF log)	None (pure cache)
Replication / HA	Replicas, Sentinel or Cluster with failover	No built-in replication (node loss = lost partition of cache)
Scaling	Cluster with 16,384 hash slots, server-side	Client-side sharding (consistent hashing) or proxy (mcrouter)
Threading	Command execution single-threaded (I/O threads in newer versions) — atomic operations	**Multi-threaded**; scales across cores on one node
Extras	Pub/sub, Lua scripts, transactions, TTL per key, streams	Very low overhead per item; simple operations
Choose when	You need structures, persistence options, HA, atomic counters/leaderboards, queues	A simple, huge, multi-core, volatile cache of blobs

Spreading keys over many cache nodes

- **Naïve** `hash(key) mod N`: adding or removing a node remaps almost every key → mass cache misses → avalanche on the database.
- **Consistent hashing** (Chapter 14): nodes and keys on a ring; adding a node moves only ~1/N of the keys; **virtual nodes** balance load. Used by Memcached clients and Dynamo-style systems.
- **Hash slots** (Redis Cluster): 16,384 fixed slots mapped to nodes; moving slots rebalances; multi-key operations need keys in the same slot (hash tags `{user42}:cart`).
- **Proxy-based** (Envoy, twemproxy, mcrouter): clients stay simple; the proxy shards, pools connections and fails over.

Operating a distributed cache

- **Sizing**: memory = number of hot keys × (key + value + ~50–100 B overhead) × ~1.5 for fragmentation; keep ≥ 20–30% free. Choose the cache size to hold the **hot set**, not all data.
- **Network cost**: every lookup is a round trip (~0.5 ms). **Batch/pipeline** (`MGET`, pipelines) instead of 100 sequential gets; keep values small; use efficient serialisation (MessagePack/Protobuf) and compress large values.

- **Availability**: replicas + automatic failover; multi-AZ; client timeouts of a few ms (a slow cache is worse than no cache — fall back to the DB or fail fast); connection pooling.
- **Monitor**: hit ratio, evictions, memory, latency percentiles, connected clients, replication lag, slow log, key-space size, hot keys. A falling hit ratio is an early-warning signal.

11.8 Common cache patterns in system design

Pattern	How it works	Key detail
Session store	`session:<id>` → JSON, TTL = idle timeout; makes app servers stateless (Chapter 5)	Use sliding expiry; persistence/replication so a restart doesn't log everyone out
Page / fragment cache	Cache rendered HTML or API responses by URL + variant	Vary key on language/device; avoid caching personalised parts
Feed / timeline cache	**Fan-out on write**: when a user posts, push the post id into each follower's Redis list/sorted set; read = one fetch. Celebrity accounts use fan-out on read (hybrid)	Store ids, hydrate entities from the object cache; cap list length
Leaderboard / counters	Redis sorted sets, `INCR`; periodic flush to DB (write-behind)	Atomic ops; accept small loss on crash
Rate limiter	`INCR` + `EXPIRE` per client/window; token bucket via Lua (Chapter 14)	Atomic; fail-open vs fail-closed decision
Look-aside for hot reads	Product/profile objects keyed by id with TTL + delete-on-write	The default pattern
Negative cache	Remember 'not found' / 'no permission' for a short time	Stops penetration
Request-result memoisation	Cache expensive computations (reports, recommendations, search) by input hash	Include code/model version in the key

11.9 Disadvantages & when not to cache

- **Need to maintain consistency** between caches and the source of truth (such as the database) through cache invalidation.
- **Cache invalidation is a difficult problem**; there is additional complexity associated with **when to update the cache**.
- **Need to make application changes** such as adding Redis or Memcached (client code, serialisation, key design, fallbacks).
- Extra infrastructure to run, secure, monitor and pay for; new failure modes (stampede, avalanche, cold start); harder debugging ("which layer served this?").

> ✖ **COMMON MISTAKE — Don't cache when…**
>
> • **Data is rarely re-read** (low hit ratio — you pay the overhead for nothing) or is **write-heavy / changes constantly**. The backend is already fast and cheap (a primary-key lookup in memory-resident DB). You need **strong consistency** with no staleness (balances, stock reservation, permissions). The data is **huge or highly personalised** so the hot set doesn't fit. Fix the slow query or index first — caching hides problems.

11.10 Worked examples

Example A — Do we need a cache? How big? (profile service: 20,000 reads/s, 50 M users, 1 KB per profile, DB sustains 3,000 reads/s)

Step	Calculation	Result
Required hit ratio	DB budget 3,000 of 20,000 req/s → misses ≤ 15%	**≥ 85%** (aim 95% for headroom → DB sees 1,000/s)
Hot set (80/20 rule)	20% of 50 M users = 10 M profiles × 1 KB	10 GB raw
With overhead ×1.5	10 GB × 1.5	**≈ 15 GB** → e.g., 3 shards × 8 GB usable + a replica each, or 2 × 16 GB nodes
Latency	0.95 × 1 ms + 0.05 × 21 ms	**≈ 2.0 ms** average (vs 20 ms without cache)

Example B — Stampede arithmetic (hot key: 5,000 req/s; DB rebuild takes 200 ms)

When the key expires, requests arriving during the rebuild all miss: 5,000 × 0.2 s = **≈ 1,000 simultaneous DB queries** for the same data. With single-flight/lease, the DB runs **1** query and 999 requests wait ~200 ms; with stale-while-revalidate nobody waits at all.

Example C — Avalanche and jitter (1 M keys all loaded at deploy with TTL = 3600 s)

Without jitter all 1 M keys expire in the same second → up to 1 M misses at once. With ±10% jitter, expiry spreads over 720 s: ≈ 1,400 refreshes/s — a manageable background load instead of a spike.

Example D — Which write strategy?

Feature	Strategy	Reason
User profile (read-heavy, edited rarely)	Cache-aside + delete on write, TTL 10 min	Simple, resilient, tolerates brief staleness
Shopping cart (read-your-writes critical)	Write-through (or update + read from cache)	User must see their own change immediately
Video view counter (millions of increments)	Write-behind (batch flush every few seconds)	Reduces DB writes ~1000×; small loss acceptable
Trending list for the homepage	Refresh-ahead / scheduled rebuild	Predictable hot key; never expire under load
Audit/bank ledger	**No cache** (or read-through from primary)	Strong consistency required

11.11 Practice questions (try before looking)

P1. Hit ratio is 95%, cache 1 ms, DB 20 ms (a miss also pays the 1 ms lookup). Average latency, and DB load at 10,000 req/s?

Answer: 0.95×1 + 0.05×21 = **2.0 ms**; DB load = 5% × 10,000 = **500 req/s**.

P2. After a profile update, users sometimes still see the old name for an hour. Likely cause and fix?

Answer: No invalidation on write; only TTL (1 h). Delete (or update) the cache key after the DB commit, keep a shorter TTL as a safety net.

P3. Your site is attacked with requests for random non-existent product ids and the DB melts, although the cache is healthy. Why and how to stop it?

Answer: Cache penetration: misses are never cached. Cache negative results briefly, add a Bloom filter/ID validation, and rate-limit.

P4. Redis node holding 25% of keys dies and is replaced by an empty node. What happens and what do you do?

Answer: Hit ratio drops → 25% of reads hit the DB (cold start). Use replicas with failover, pre-warm, protect DB with limits/coalescing; with consistent hashing/slots only that node's keys are affected.

P5. Why is 'update cache then DB' (or two concurrent writers updating both) risky?

Answer: Ordering races leave the cache and DB with different values; if the DB write fails, the cache holds data that never existed. Prefer DB first, then delete the key.

P6. You expect 40,000 reads/s of 2 KB objects from a cache cluster on 1 Gbps links. Is network a limit?

Answer: 40,000 × 2 KB = 80 MB/s ≈ 640 Mbps — close to a 1 Gbps link (≈ 125 MB/s): spread over several nodes/NICs, compress, or use a local L1 cache.

Interview Q&A (30-second answers)

Q1. Why use a cache and where do you put it?

To cut latency and backend load by serving repeated reads from memory. Layers: browser, CDN, reverse proxy, in-process, distributed (Redis/Memcached), database buffer pool.

Q2. Explain cache-aside.

App checks the cache; on a miss loads from the DB, writes to the cache with a TTL, returns. Lazy, resilient, but a miss costs three trips and data can be stale.

Q3. Write-through vs write-behind?

Write-through: cache writes to the DB synchronously — consistent, slower writes. Write-behind: cache acknowledges, writes later asynchronously — fastest writes, risk of data loss and more complexity.

Q4. What is refresh-ahead?

Refresh hot entries before they expire so users never see a miss; helps only if you predict demand correctly.

Q5. How do you invalidate a cache?

TTL as baseline plus delete-on-write, event/CDC-driven invalidation, versioned keys or tags; prefer entity-level caches over query-level ones.

Q6. What is a cache stampede and how do you prevent it?

Many requests rebuild an expired hot key simultaneously. Use single-flight/locks, stale-while-revalidate, refresh-ahead, TTL jitter.

Q7. Cache penetration vs avalanche vs hot key?

Penetration: non-existent keys bypass cache (cache nulls, Bloom filter). Avalanche: mass expiry/outage overloads DB (jitter, HA, load shedding). Hot key: one key overloads a node (L1 cache, key replication).

Q8. LRU vs LFU?

LRU evicts the least recently used (temporal locality); LFU evicts the least frequently used (stable popularity, scan-resistant).

Q9. Redis vs Memcached?

Redis: data structures, persistence, replication, cluster, atomic ops. Memcached: simple multi-threaded volatile blob cache.

Q10. When should you not use a cache?

Low reuse, write-heavy data, strict consistency, cheap backend, or when the real fix is a missing index/slow query.

Chapter 11 Cheat Sheet

⚡ **QUICK RECALL** — **Revise this in 2 minutes**

- **Why:** memory ≫ disk/network speed; locality (80/20); absorbs spikes and skew. Think in **miss rate**; cache is never the source of truth.
- **Layers:** client → CDN → reverse proxy (Varnish/Nginx) → app (in-process L1) → distributed cache (Redis/Memcached L2) → DB buffer pool. Avoid file-based caches.
- **Granularity:** row, query (hard to invalidate), **object** (best), rendered HTML. Cache: sessions, pages, activity streams, user graph, catalogue.
- **Cache-aside** (lazy; 3 trips on miss; stale until TTL; Memcached style) • **Read-through** • **Write-through** (sync, no stale, slow writes) • **Write-behind** (async, fast, data-loss risk, complex) • **Refresh-ahead** (predict hot keys) • **Write-around**.
- **Default:** cache-aside + DB write, then **delete** key + TTL (+ jitter). Avoid updating cache and DB concurrently (race).
- **Eviction:** LRU (hash map + linked list), LFU, TinyLFU, TTL; Redis allkeys-lru/lfu, volatile-ttl, noeviction.
- **Failures:** stampede (single-flight, SWR, jitter) • penetration (cache nulls, Bloom) • avalanche (jitter, HA, load-shed) • hot key (L1, replicate) • cold start (warm-up).
- **Distributed:** consistent hashing / 16,384 hash slots; never `mod N`; replicas + failover; pipeline/batch calls; watch hit ratio, evictions, p99.
- **Redis** = structures + persistence + HA; **Memcached** = simple, multi-threaded, volatile.
- **Disadvantages:** consistency via invalidation, complexity of when to update, app changes, extra infra. Don't cache low-reuse, write-heavy or strongly consistent data.
- **Numbers:** avg = hit×1 ms + miss×21 ms; 99% hits ⇒ 10× less DB load than 90%; size = hot set × 1.5.

AMJESH https://singhamjesh.github.io/